

Computer Vision Lab 06 - Code Cheat Sheet

How to use this sheet in the coding exam

For every code problem, identify these four things first:

- **INPUT:** image/video/signal and required format (BGR, grayscale, descriptors, etc.)
- **OPERATION:** the OpenCV/NumPy/wavelet function that actually performs the CV operation
- **OUTPUT:** edge map, detected lines/circles, keypoints/descriptors, transformed image, reconstructed signal, etc.
- **PARAMETERS:** the values most likely to be changed by the question

Workflow rule

```

What do they want to FIND?
    ↓
Which technique finds it?
    ↓
What INPUT does that technique need?
    ↓
Add only the necessary preprocessing/dependencies
    ↓
What should I DO with the result?
  
```

Lab 06 topic map

```

Wavelet denoising/anomaly detection -> wavelet decomposition -> threshold ->
reconstruction -> residual
Object boundaries / edges           -> Canny / Sobel / LoG
Known line structures                -> Canny -> HoughLinesP
Known circles                       -> blur -> HoughCircles
Object recognition                  -> SIFT -> descriptors -> matching ->
RANSAC/Homography -> location
Panorama stitching                  -> SIFT -> matching -> Homography -> warpPerspective
-> stitch
Video processing                    -> VideoCapture -> read frames -> process each frame
Object boundary security             -> Canny -> contours -> boundingRect -> zone test
  
```

Important: The Lab 06 manual contains the detailed standalone code for Sobel, Canny, LoG, and SIFT. Hough is explained conceptually in the manual; the standalone Hough line/circle code below is included as an **exam/task extension**, because the Lab 06 Tasks explicitly require Hough line/circle implementations. The wavelet task likewise needs a concrete implementation even though the manual's wavelet section is mainly conceptual.

PART A - CODE FROM LAB 6 MANUAL

1. Sobel Edge Detection

Purpose

Detect intensity changes in the horizontal and vertical directions and combine them into an overall gradient magnitude.

Manual code - commented for exam use

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("image.jpeg")

# Convert BGR to RGB for correct Matplotlib display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert the image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Compute Sobel gradient in the X direction
sobel_x = cv2.Sobel(
    gray,
    cv2.CV_64F,
    1,
    0,
    ksize=3
)
```

```
# Compute Sobel gradient in the Y direction
sobel_y = cv2.Sobel(
    gray,
    cv2.CV_64F,
    0,
    1,
    ksize=3
)

# Calculate overall gradient magnitude
magnitude = np.sqrt(
    sobel_x**2 + sobel_y**2
)

# Convert magnitude to uint8 and clip values to display range 0-255
magnitude = np.uint8(
    np.clip(magnitude, 0, 255)
)

# Create a figure for displaying the results
plt.figure(figsize=(12, 8))

# Display original image
plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

# Display X-direction Sobel response
plt.subplot(2, 2, 2)
plt.imshow(sobel_x, cmap="gray")
plt.title("Sobel X")
plt.axis("off")

# Display Y-direction Sobel response
plt.subplot(2, 2, 3)
plt.imshow(sobel_y, cmap="gray")
plt.title("Sobel Y")
plt.axis("off")

# Display combined gradient magnitude
plt.subplot(2, 2, 4)
plt.imshow(magnitude, cmap="gray")
plt.title("Sobel Edge Magnitude")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()
```

Line meanings

- `cv2.imread(...)` -> reads the image.
- `cv2.cvtColor(..., cv2.COLOR_BGR2RGB)` -> converts BGR to RGB so Matplotlib shows normal colors.
- `cv2.cvtColor(..., cv2.COLOR_BGR2GRAY)` -> converts the image to grayscale.
- `cv2.Sobel(..., 1, 0, ...)` -> first derivative in X direction.
- `cv2.Sobel(..., 0, 1, ...)` -> first derivative in Y direction.
- `cv2.CV_64F` -> use 64-bit floating point so negative gradient values are preserved.
- `ksize=3` -> use a 3x3 Sobel kernel.
- `np.sqrt(...)` -> combines X/Y gradients into gradient magnitude.
- `np.clip(..., 0, 255)` -> limits values to valid display range.
- `np.uint8(...)` -> converts the result to 8-bit unsigned integers.
- `plt.subplot(2,2,i)` -> creates one position in a 2x2 display layout.

Function / argument dictionary

```
cv2.imread(filename, flags=None)
```

- `filename` = image path/name.
- `flags` = optional read mode.

```
cv2.cvtColor(src, code)
```

- `src` = input image.
- `code` = conversion code, e.g. `cv2.COLOR_BGR2RGB` , `cv2.COLOR_BGR2GRAY` .

```
cv2.Sobel(src, ddepth, dx, dy, ksize=1)
```

- `src` = input grayscale image.
- `ddepth` = output depth; manual uses `cv2.CV_64F`.
- `dx` = derivative order in X direction.
- `dy` = derivative order in Y direction.
- `ksize` = Sobel kernel size; manual uses `3`.

`np.sqrt(a)`

- `a` = value/array whose square root is required.

`np.clip(a, min_value, max_value)`

- `a` = input array.
- `min_value` = lower limit.
- `max_value` = upper limit.

`np.uint8(a)`

- `a` = array/value converted to 8-bit unsigned integer type.

`plt.figure(figsize=(w,h))`

- `figsize` = figure width and height.

`plt.subplot(rows, cols, index)`

- `rows` = number of rows.
- `cols` = number of columns.
- `index` = current position.

`plt.imshow(image, cmap=None)`

- `image` = image/array to display.
- `cmap` = display colormap; manual uses `"gray"` for grayscale outputs.

`plt.title(text)` -> sets plot title.

`plt.axis(setting)` -> manual uses "off" to hide axes.

`plt.tight_layout()` -> improves spacing.

`plt.show()` -> displays the figure.

Exam modification

If the question asks for only X-gradient:

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
```

If it asks for only Y-gradient:

```
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
```

If it asks for total gradient:

```
magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
```

2. Canny Edge Detection

Purpose

Detect thin, useful edges using Gaussian smoothing, gradients, non-maximum suppression, and hysteresis thresholding.

Manual code - commented for exam use

```
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("image.jpeg")

# Convert BGR to RGB for Matplotlib display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
```

```
# Convert image to grayscale because Canny uses intensity information
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Smooth the grayscale image to reduce noise
blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    1.4
)

# Detect edges using lower and upper hysteresis thresholds
edges = cv2.Canny(
    blurred,
    50,
    150
)

# Create display figure
plt.figure(figsize=(12, 8))

# Original image
plt.subplot(2, 2, 1)
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

# Grayscale image
plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# Gaussian-smoothed image
plt.subplot(2, 2, 3)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred")
plt.axis("off")

# Final Canny edge map
plt.subplot(2, 2, 4)
plt.imshow(edges, cmap="gray")
plt.title("Canny Edges")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Line meanings

- `GaussianBlur` reduces noise before edge detection.
- `(5,5)` is the Gaussian kernel size.
- `1.4` is sigma in the manual code.
- `Canny(..., 50, 150)` uses `50` as the lower threshold and `150` as the upper threshold.
- The output `edges` is a binary-style edge map.

Function / argument dictionary

```
cv2.GaussianBlur(src, ksize, sigmaX)
```

- `src` = input image.
- `ksize` = Gaussian kernel size, e.g. `(5,5)` .
- `sigmaX` = standard deviation in X direction; manual uses `1.4` .

```
cv2.Canny(image, threshold1, threshold2)
```

- `image` = input image, normally grayscale.
- `threshold1` = lower hysteresis threshold.
- `threshold2` = upper hysteresis threshold.
- output = edge map.

Exam modification

Only threshold change:

```
edges = cv2.Canny(gray, 100, 200)
```

If the question emphasizes noise, include Gaussian blur first:

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)  
edges = cv2.Canny(blurred, 100, 200)
```

3. Laplacian of Gaussian (LoG)

Purpose

Smooth the image with a Gaussian filter and then apply the Laplacian second derivative. LoG edges are associated with zero crossings of the second derivative.

Manual code - commented for exam use

```
import cv2
import matplotlib.pyplot as plt

# Read the image
image = cv2.imread("image.jpeg")

# Convert BGR to RGB for Matplotlib display
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Convert image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Smooth the image to reduce noise before the Laplacian
blurred = cv2.GaussianBlur(
    gray,
    (5, 5),
    1.4
)

# Apply the Laplacian second-derivative operator
laplacian = cv2.Laplacian(
    blurred,
    cv2.CV_64F
)

# Convert Laplacian response into a display-friendly 8-bit image
laplacian_abs = cv2.convertScaleAbs(
    laplacian
)

# Create display figure
plt.figure(figsize=(12, 8))

# Original image
plt.subplot(2, 2, 1)
```

```
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")

# Grayscale image
plt.subplot(2, 2, 2)
plt.imshow(gray, cmap="gray")
plt.title("Grayscale")
plt.axis("off")

# Gaussian-smoothed image
plt.subplot(2, 2, 3)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred")
plt.axis("off")

# Laplacian response
plt.subplot(2, 2, 4)
plt.imshow(laplacian_abs, cmap="gray")
plt.title("Laplacian of Gaussian")
plt.axis("off")

plt.tight_layout()
plt.show()
```

Line meanings

- Gaussian blur comes before Laplacian to reduce noise.
- `cv2.Laplacian` computes the second derivative.
- `cv2.CV_64F` preserves positive and negative Laplacian values internally.
- `cv2.convertScaleAbs` produces an absolute 8-bit result for display.

Function / argument dictionary

`cv2.Laplacian(src, ddepth)`

- `src` = input image.
- `ddepth` = output depth; manual uses `cv2.CV_64F`.

`cv2.convertScaleAbs(src, alpha=1, beta=0)`

- `src` = input array.
- `alpha` = optional scale factor.
- `beta` = optional offset.
- output = absolute 8-bit image.

Exam recognition

LoG -> Gaussian Blur -> Laplacian -> zero-crossing idea -> edges

4. SIFT Feature Extraction

Purpose

Find distinctive local features and produce numerical descriptors that can later be matched between images.

Manual code - commented for exam use

```
import cv2

# Convert the input image to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Create the SIFT detector
sift = cv2.SIFT_create()

# Detect keypoints and compute their SIFT descriptors
keypoints, descriptors = sift.detectAndCompute(
    gray,
    None
)

# Draw the detected keypoints on the RGB image
image_keypoints = cv2.drawKeypoints(
    image_rgb,
    keypoints,
    None,
```

```

        flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
    )

    # Print number of detected keypoints
    print("Number of keypoints:", len(keypoints))

    # Print descriptor matrix shape
    print("Descriptor shape:", descriptors.shape)

```

`image` , `image_rgb` are assumed to have already been loaded/created as in the earlier manual code block.

Line meanings

- `cv2.SIFT_create()` -> creates a SIFT feature detector/descriptor extractor.
- `detectAndCompute(gray, None)` -> finds keypoints and computes descriptors.
- `keypoints` = detected feature locations and related keypoint information.
- `descriptors` = numerical description of each keypoint.
- Standard SIFT descriptors are 128-dimensional, so the descriptor matrix has the general form `(number_of_keypoints, 128)` .
- `drawKeypoints` visualizes the detected keypoints.

Function / argument dictionary

`cv2.SIFT_create()`

- creates SIFT object.
- common optional parameters exist in OpenCV, but manual uses the default constructor.

`sift.detectAndCompute(image, mask)`

- `image` = grayscale image in the manual.
- `mask` = optional mask; manual uses `None` .
- returns `(keypoints, descriptors)` .

`cv2.drawKeypoints(image, keypoints, outImage=None, color=None, flags=0)`

- `image` = image on which to draw.
- `keypoints` = SIFT keypoints.
- `outImage` = destination; manual uses `None` .
- `color` = optional color.
- `flags` = drawing style; manual uses `cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS` .

Exam recognition

SIFT -> keypoints + descriptors

Do NOT confuse:

keypoints = where the features are

descriptors = numerical representation used for matching

PART B - HOUGH TRANSFORM CODE FOR EXAM / TASKS

The Lab 06 manual explains Hough line/circle detection conceptually. These are the compact OpenCV implementations you need for the Lab 06 tasks.

5. Hough Line Detection

Purpose

Detect line segments from an edge image.

Workflow

Image -> Grayscale -> Blur -> Canny -> HoughLinesP -> Draw Lines

Code

```
import cv2
import numpy as np

# Read input image
image = cv2.imread("image.jpg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Reduce noise
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Detect edges
edges = cv2.Canny(blurred, 50, 150)

# Detect line segments using the probabilistic Hough transform
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    threshold=80,
    minLineLength=50,
    maxLineGap=10
)

# Copy image for drawing
output = image.copy()

# Draw every detected line
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(
            output,
            (x1, y1),
            (x2, y2),
            (0, 255, 0),
            2
        )

cv2.imshow("Detected Lines", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

```
cv2.HoughLinesP(image, rho, theta, threshold, minLineLength=None, maxLineGap=None)
```

- `image` = edge image, typically Canny output.
- `rho` = distance resolution; commonly `1` pixel.
- `theta` = angular resolution in radians; `np.pi/180` = 1 degree.
- `threshold` = minimum accumulator votes.
- `minLineLength` = minimum accepted line length.
- `maxLineGap` = maximum gap allowed when joining line segments.

```
cv2.line(img, pt1, pt2, color, thickness)
```

- `img` = image to draw on.
- `pt1` = first endpoint `(x1,y1)`.
- `pt2` = second endpoint `(x2,y2)`.
- `color` = BGR color tuple.
- `thickness` = line thickness.

Exam recognition

Question says: lanes, screen boundaries, straight lines.

Think:

```
Canny -> HoughLinesP -> draw lines
```

6. Hough Circle Detection

Purpose

Detect circular objects such as coins.

Workflow

Image -> Grayscale -> Blur -> HoughCircles -> Draw / Count Circles

Code

```
import cv2
import numpy as np

# Read image
image = cv2.imread("coins.jpg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Smooth the image to reduce noise
blurred = cv2.GaussianBlur(gray, (9, 9), 2)

# Detect circles
circles = cv2.HoughCircles(
    blurred,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

# Count detected circles
if circles is not None:
    circles = np.uint16(np.around(circles))
    print("Number of circles:", len(circles[0]))

    # Draw each detected circle
    for x, y, r in circles[0]:
        cv2.circle(image, (x, y), r, (0, 255, 0), 2)
        cv2.circle(image, (x, y), 2, (0, 0, 255), 3)
else:
    print("Number of circles: 0")
```

```
cv2.imshow("Detected Circles", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Function / argument dictionary

```
cv2.HoughCircles(image, method, dp, minDist, param1=None, param2=None, minRadius=0,
maxRadius=0)
```

- `image` = input grayscale image.
- `method` = circle-detection method; manual extension uses `cv2.HOUGH_GRADIENT`.
- `dp` = inverse ratio of accumulator resolution to image resolution.
- `minDist` = minimum distance between detected circle centers.
- `param1` = higher Canny threshold used internally by the Hough method.
- `param2` = accumulator threshold used to accept/reject circles.
- `minRadius` = minimum circle radius.
- `maxRadius` = maximum circle radius.

`np.around(a)` -> rounds values to nearest integer.

Exam recognition

Question says: coins, circular objects, circles.

Think:

```
Blur -> HoughCircles -> count/draw circles
```

PART C - MATCHING / HOMOGRAPHY CODE BANK

7. SIFT + Descriptor Matching + Homography

Workflow



Code

```

import cv2
import numpy as np

# Read reference and test images
reference = cv2.imread("reference.jpg")
test = cv2.imread("test.jpg")

# Convert both images to grayscale
gray_ref = cv2.cvtColor(reference, cv2.COLOR_BGR2GRAY)
gray_test = cv2.cvtColor(test, cv2.COLOR_BGR2GRAY)

# Create SIFT
sift = cv2.SIFT_create()

```

```

# Find keypoints and descriptors in both images
kp1, des1 = sift.detectAndCompute(gray_ref, None)
kp2, des2 = sift.detectAndCompute(gray_test, None)

# Create a brute-force matcher
bf = cv2.BFMatcher()

# Find the two nearest descriptor matches for each reference descriptor
matches = bf.knnMatch(des1, des2, k=2)

# Keep only reliable matches using the ratio test
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

# Homography needs at least 4 point correspondences
if len(good_matches) >= 4:

    # Points from the reference image
    src_pts = np.float32([
        kp1[m.queryIdx].pt
        for m in good_matches
    ]).reshape(-1, 1, 2)

    # Corresponding points from the test image
    dst_pts = np.float32([
        kp2[m.trainIdx].pt
        for m in good_matches
    ]).reshape(-1, 1, 2)

    # Estimate geometric transformation and reject outliers with RANSAC
    H, mask = cv2.findHomography(
        src_pts,
        dst_pts,
        cv2.RANSAC,
        5.0
    )

    print("Good matches:", len(good_matches))
    print("Homography:\n", H)

```

Function / argument dictionary

```
cv2.BFMatcher(normType=None, crossCheck=False)
```

- creates brute-force descriptor matcher.
- for standard SIFT descriptors, OpenCV can use the default appropriate norm.

```
bf.knnMatch(queryDescriptors, trainDescriptors, k)
```

- `queryDescriptors` = descriptors from first image.
- `trainDescriptors` = descriptors from second image.
- `k` = number of nearest matches requested; use `2` for ratio-test pattern.

`m.queryIdx` -> index of the matched descriptor/keypoint in the first image.

`m.trainIdx` -> index of the matched descriptor/keypoint in the second image.

`m.distance` -> descriptor distance; smaller generally means a better match.

```
cv2.findHomography(srcPoints, dstPoints, method=0, ransacReprojThreshold=3.0)
```

- `srcPoints` = points from source image.
- `dstPoints` = corresponding points in destination image.
- `method` = estimation method; `cv2.RANSAC` rejects outliers.
- `ransacReprojThreshold` = RANSAC reprojection tolerance.
- returns homography matrix `H` and an inlier mask.

Key memory

Matching = which features correspond?

Homography = how are the two images geometrically related?

RANSAC = which matches are consistent with that geometry?

8. Bounding an Object After Homography

```
# Get reference image dimensions
h, w = gray_ref.shape
```

```
# Define the four corners of the reference object/image
```

```

corners = np.float32([
    [0, 0],
    [w, 0],
    [w, h],
    [0, h]
]).reshape((-1, 1, 2))

# Transform reference corners into the test-image coordinate system
transformed_corners = cv2.perspectiveTransform(
    corners,
    H
)

# Draw the transformed object boundary
cv2.polylines(
    test,
    [np.int32(transformed_corners)],
    True,
    (0, 255, 0),
    3
)

```

Function / argument dictionary

`cv2.perspectiveTransform(src, M)`

- `src` = points to transform.
- `M` = transformation matrix, here the homography `H`.

`cv2.polylines(img, pts, isClosed, color, thickness)`

- `img` = destination image.
- `pts` = list of point arrays.
- `isClosed` = `True` to close the polygon.
- `color` = BGR tuple.
- `thickness` = line thickness.

Exam recognition

Question says: recognize object + draw bounding box.

Think:

```
SIFT -> matching -> RANSAC/Homography -> perspectiveTransform -> polylines
```

PART D - LAB 6 TASKS

Task 1 - Computer Screen Detection

What the task wants

Use **Hough Line Transformation** to identify the boundaries of computer screens in a computer lab and use the detected structure to help identify anomalies such as missing screens.

Workflow

```
Computer lab image
  ↓
  Grayscale
  ↓
  Gaussian Blur
  ↓
  Canny
  ↓
  HoughLinesP
  ↓
  Detected line segments
  ↓
  Draw / analyze screen boundaries
  ↓
  Compare expected screen arrangement for anomalies
```

Complete solution

```
import cv2
import numpy as np

# Read the computer-lab image
image = cv2.imread("computer_lab.jpg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Reduce small noise before edge detection
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Detect edges of screen boundaries
edges = cv2.Canny(blurred, 50, 150)

# Detect straight line segments
lines = cv2.HoughLinesP(
    edges,
    1,
    np.pi / 180,
    threshold=80,
    minLineLength=50,
    maxLineGap=10
)

# Create a copy for visualization
output = image.copy()

# Draw detected line segments
if lines is not None:
    for line in lines:
        x1, y1, x2, y2 = line[0]
        cv2.line(
            output,
            (x1, y1),
            (x2, y2),
            (0, 255, 0),
            2
        )

cv2.imshow("Screen Boundaries", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Functions used

- `cv2.imread(filename, flags=None)`
- `cv2.cvtColor(src, code)`
- `cv2.GaussianBlur(src, ksize, sigmaX)`
- `cv2.Canny(image, threshold1, threshold2)`
- `cv2.HoughLinesP(image, rho, theta, threshold, minLineLength, maxLineGap)`
- `cv2.line(img, pt1, pt2, color, thickness)`
- `cv2.imshow(windowName, image)`
- `cv2.waitKey(delay)`
- `cv2.destroyAllWindows()`

Main exam parameters

- Canny low/high threshold.
- Hough threshold .
- minLineLength .
- maxLineGap .

Important note

Hough returns lines; the problem of deciding that a whole screen is "missing" requires an additional interpretation rule. The task sheet specifies the anomaly goal but does not prescribe a specific screen-count/grouping algorithm. In an exam, implement the requested Hough boundary detection first, then add the simplest reasonable comparison rule if Sir explicitly asks for the anomaly check.

Task 2 - Computer-Lab Asset Tracking using SIFT

What the task wants

Use SIFT to distinguish/recognize individual assets such as monitors/keyboards by comparing a reference asset against a test image.

Workflow

```
Reference asset + Test image
↓
SIFT
↓
Keypoints + descriptors
↓
BF matching
↓
Ratio test
↓
Good matches
↓
RANSAC / Homography
↓
Asset location
```

Complete solution

```
import cv2
import numpy as np

# Read reference asset and lab/test image
reference = cv2.imread("reference.jpg")
test = cv2.imread("test.jpg")

# Convert both images to grayscale
gray_ref = cv2.cvtColor(reference, cv2.COLOR_BGR2GRAY)
gray_test = cv2.cvtColor(test, cv2.COLOR_BGR2GRAY)
```

```
# Create SIFT detector
sift = cv2.SIFT_create()

# Detect features and compute descriptors in both images
kp1, des1 = sift.detectAndCompute(gray_ref, None)
kp2, des2 = sift.detectAndCompute(gray_test, None)

# Create descriptor matcher
bf = cv2.BFMatcher()

# Find two nearest matches for each descriptor
matches = bf.knnMatch(des1, des2, k=2)

# Ratio-test filtering
good_matches = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good_matches.append(m)

# Estimate homography only when enough matches exist
if len(good_matches) >= 4:

    src_pts = np.float32([
        kp1[m.queryIdx].pt
        for m in good_matches
    ]).reshape(-1, 1, 2)

    dst_pts = np.float32([
        kp2[m.trainIdx].pt
        for m in good_matches
    ]).reshape(-1, 1, 2)

    H, mask = cv2.findHomography(
        src_pts,
        dst_pts,
        cv2.RANSAC,
        5.0
    )

    if H is not None:
        h, w = gray_ref.shape
        corners = np.float32([
            [0, 0],
            [w, 0],
            [w, h],
```

```
        [0, h]
    ]).reshape(-1, 1, 2)

    object_corners = cv2.perspectiveTransform(
        corners,
        H
    )

    cv2.polylines(
        test,
        [np.int32(object_corners)],
        True,
        (0, 255, 0),
        3
    )

cv2.imshow("Recognized Asset", test)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Functions used

- cv2.imread
- cv2.cvtColor
- cv2.SIFT_create
- sift.detectAndCompute
- cv2.BFMatcher
- bf.knnMatch
- cv2.findHomography
- cv2.perspectiveTransform
- cv2.polylines
- cv2.imshow
- cv2.waitKey
- cv2.destroyAllWindows

Main exam parameters

- Ratio-test value, e.g. `0.7` .
- RANSAC reprojection threshold, e.g. `5.0` .
- Minimum number of matches before homography: commonly `4` .

Task 3 - Sensor Anomaly Detection using Wavelet Transformation

What the task wants

Use wavelet transformation to denoise sensor data, reconstruct the cleaned signal, calculate residuals, and detect unusually large residuals as anomalies.

The task statement does not prescribe an exact wavelet family, decomposition level, or anomaly threshold, so the values below are one concrete implementation rather than an “official parameter set.”

Workflow

```
Sensor data
↓
Wavelet decomposition
↓
Threshold/suppress noise coefficients
↓
Inverse wavelet reconstruction
↓
Denoised signal
↓
Residual = original - denoised
↓
Residual threshold
↓
Anomaly points
```

Complete solution

```
import numpy as np
import matplotlib.pyplot as plt
import pywt

# Load 1-D sensor readings
sensor_data = np.loadtxt("sensor_data.csv")

# Decompose signal using a Daubechies wavelet
coeffs = pywt.wavedec(
    sensor_data,
    "db4",
    level=4
)

# Estimate noise level from the highest-frequency detail coefficients
sigma = np.median(
    np.abs(coeffs[-1])
) / 0.6745

# Calculate a denoising threshold
threshold = sigma * np.sqrt(
    2 * np.log(len(sensor_data))
)

# Keep approximation coefficients and threshold detail coefficients
denoised_coeffs = [coeffs[0]]

for c in coeffs[1:]:
    denoised_coeffs.append(
        pywt.threshold(
            c,
            threshold,
            mode="soft"
        )
    )

# Reconstruct denoised signal
denoised_signal = pywt.waverec(
    denoised_coeffs,
    "db4"
)

# Match reconstructed length to original length
```

```

denoised_signal = denoised_signal[:len(sensor_data)]

# Calculate residual signal
residuals = sensor_data - denoised_signal

# Define anomaly threshold
anomaly_threshold = 2.5 * np.std(residuals)

# Mark residuals larger than the threshold
anomalies = np.abs(residuals) > anomaly_threshold

# Plot original + denoised signal
plt.figure(figsize=(12, 8))
plt.subplot(2, 1, 1)
plt.plot(sensor_data, label="Sensor Data")
plt.plot(denoised_signal, label="Denoised Signal")
plt.title("Sensor Data and Denoised Signal")
plt.legend()

# Plot residuals + anomaly points
plt.subplot(2, 1, 2)
plt.plot(residuals, label="Residuals")
plt.scatter(
    np.where(anomalies)[0],
    residuals[anomalies],
    label="Anomalies"
)
plt.title("Residuals and Detected Anomalies")
plt.legend()

plt.tight_layout()
plt.show()

```

Function / argument dictionary

`np.loadtxt(fname)`

- `fname` = text/CSV-like numeric data file.

`pywt.wavedec(data, wavelet, level=None)`

- `data` = input signal.
- `wavelet` = wavelet family/name; example `"db4"`.

- `level` = decomposition level.
- returns wavelet coefficient arrays.

```
pywt.threshold(data, value, mode="soft")
```

- `data` = coefficient array.
- `value` = threshold.
- `mode` = thresholding type, here `"soft"` .

```
pywt.waverec(coeffs, wavelet)
```

- `coeffs` = wavelet coefficient list.
- `wavelet` = same wavelet used for decomposition.
- returns reconstructed signal.

```
np.median(a) -> median value.
```

```
np.abs(a) -> absolute value element-wise.
```

```
np.std(a) -> standard deviation.
```

```
np.where(condition) -> indices satisfying the condition.
```

Main exam variables to change

- wavelet family (`"db4"` here).
- decomposition `level` .
- coefficient threshold.
- anomaly threshold multiplier (`2.5` here).

Core memory

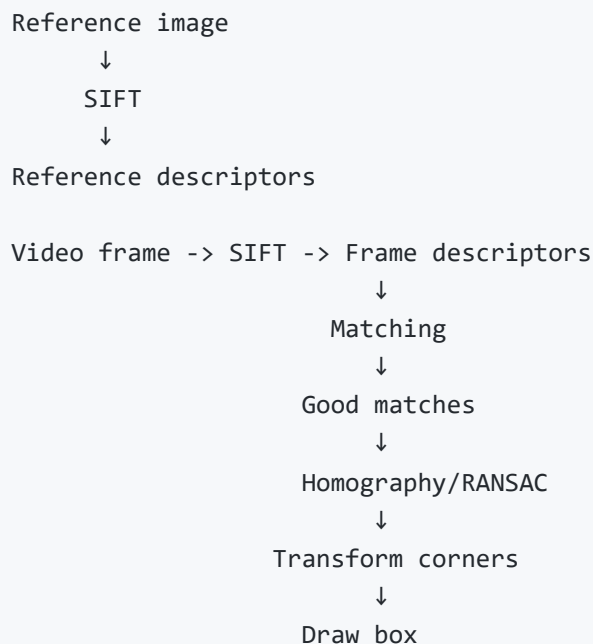
```
Wavelet -> denoise -> reconstruct -> residual -> threshold residual -> anomalies
```

Task 4 - Object Recognition from Video using SIFT

What the task wants

Recognize a reference object inside a video, even when scale/orientation changes occur, and draw a bounding box around the detected object.

Workflow



Complete solution

```
import cv2
import numpy as np

# Read reference object
reference = cv2.imread("reference.jpg")
gray_ref = cv2.cvtColor(reference, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
```

```
sift = cv2.SIFT_create()

# Detect reference keypoints and descriptors
kp_ref, des_ref = sift.detectAndCompute(
    gray_ref,
    None
)

# Create descriptor matcher
bf = cv2.BFMatcher()

# Open test video
cap = cv2.VideoCapture("test_video.mp4")

while True:

    # Read one frame
    ret, frame = cap.read()

    # Stop when video ends / frame cannot be read
    if not ret:
        break

    # Convert current frame to grayscale
    gray_frame = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # Detect SIFT features in current frame
    kp_frame, des_frame = sift.detectAndCompute(
        gray_frame,
        None
    )

    # Only attempt matching if descriptors exist
    if des_frame is not None:

        # Match each reference descriptor to its 2 nearest frame descriptors
        matches = bf.knnMatch(
            des_ref,
            des_frame,
            k=2
        )

        # Keep good matches using the ratio test
```

```

good = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good.append(m)

# Need enough correspondences for homography
if len(good) >= 4:

    # Reference points
    src_pts = np.float32([
        kp_ref[m.queryIdx].pt
        for m in good
    ]).reshape(-1, 1, 2)

    # Current-frame points
    dst_pts = np.float32([
        kp_frame[m.trainIdx].pt
        for m in good
    ]).reshape(-1, 1, 2)

    # Estimate homography with RANSAC
    H, mask = cv2.findHomography(
        src_pts,
        dst_pts,
        cv2.RANSAC,
        5.0
    )

    if H is not None:

        # Reference image corners
        h, w = gray_ref.shape
        corners = np.float32([
            [0, 0],
            [w, 0],
            [w, h],
            [0, h]
        ]).reshape(-1, 1, 2)

        # Transform corners to frame coordinates
        box = cv2.perspectiveTransform(
            corners,
            H
        )

        # Draw detected object boundary

```

```

        cv2.polylines(
            frame,
            [np.int32(box)],
            True,
            (0, 255, 0),
            3
        )

    # Display current processed frame
    cv2.imshow("Object Recognition", frame)

    # Press q to quit
    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

# Release video and close windows
cap.release()
cv2.destroyAllWindows()

```

Function / argument dictionary

`cv2.VideoCapture(source)`

- `source` = video filename or camera index.

`cap.read()`

- returns `(ret, frame)`.
- `ret` = whether a frame was successfully read.
- `frame` = current image frame.

`cv2.waitKey(delay)`

- `delay` = milliseconds to wait for a key event.
- in the video loop, `1` is common.

`cap.release()` -> closes the video source.

Exam recognition

Question says: live/test video + recognize a known object.

Think:

```
VideoCapture -> read frame -> SIFT -> match -> Homography -> bounding box
```

Task 5 - Create a Panoramic Image

What the task wants

Use SIFT keypoints and descriptor matching to align overlapping images and create a panorama.

Workflow

```
Image 1 + Image 2
  ↓
  SIFT
  ↓
Keypoints + descriptors
  ↓
  Matching
  ↓
Good correspondences
  ↓
Homography/RANSAC
  ↓
warpPerspective
  ↓
Combine / stitch
  ↓
Panorama
```

Complete two-image implementation

```
import cv2
import numpy as np

# Read the two overlapping images
img1 = cv2.imread("image1.jpg")
```

```
img2 = cv2.imread("image2.jpg")

# Convert both images to grayscale
gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)

# Create SIFT detector
sift = cv2.SIFT_create()

# Find keypoints and descriptors
kp1, des1 = sift.detectAndCompute(gray1, None)
kp2, des2 = sift.detectAndCompute(gray2, None)

# Create matcher
bf = cv2.BFMatcher()

# Match image 2 descriptors to image 1 descriptors
matches = bf.knnMatch(des2, des1, k=2)

# Keep reliable matches
good = []
for m, n in matches:
    if m.distance < 0.7 * n.distance:
        good.append(m)

# Need at least four pairs for homography
if len(good) >= 4:

    # Source points are from image 2
    src_pts = np.float32([
        kp2[m.queryIdx].pt
        for m in good
    ]).reshape(-1, 1, 2)

    # Destination points are in image 1
    dst_pts = np.float32([
        kp1[m.trainIdx].pt
        for m in good
    ]).reshape(-1, 1, 2)

    # Estimate transformation
    H, mask = cv2.findHomography(
        src_pts,
        dst_pts,
        cv2.RANSAC,
        5.0
```

```

)

# Get image dimensions
h1, w1 = img1.shape[:2]
h2, w2 = img2.shape[:2]

# Warp image 2 into image 1 coordinate system
panorama = cv2.warpPerspective(
    img2,
    H,
    (w1 + w2, max(h1, h2))
)

# Place image 1 into the left side of the panorama
panorama[0:h1, 0:w1] = img1

cv2.imshow("Panorama", panorama)
cv2.waitKey(0)

cv2.destroyAllWindows()

```

Function / argument dictionary

`image.shape[:2]`

- returns (height, width) for a color image.

`cv2.warpPerspective(src, M, dsize)`

- `src` = image being warped.
- `M` = perspective transformation matrix, here `H`.
- `dsize` = output size (width, height).

Key memory

`findHomography` -> calculate how the images align
`warpPerspective` -> actually transform/warp the image

Exam recognition

Question says: panorama / image stitching.

Think:

SIFT -> matching -> Homography -> warpPerspective -> stitch

Task 6 - Lane Detection using Hough Line Transformation

What the task wants

Detect lane markings in a road image using Hough line transformation.

Workflow

```
Road image
↓
Grayscale
↓
Blur
↓
Canny
↓
Region of interest (ROI)
↓
HoughLinesP
↓
Lane line segments
↓
Draw lines
```

Complete solution

```
import cv2
import numpy as np

# Read road image
image = cv2.imread("road.jpg")
```

```
# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Reduce noise
blurred = cv2.GaussianBlur(gray, (5, 5), 0)

# Detect edges
edges = cv2.Canny(blurred, 50, 150)

# Image dimensions
height, width = edges.shape

# Create empty ROI mask
mask = np.zeros_like(edges)

# Keep the road/lane region and ignore irrelevant image areas
polygon = np.array([[
    (0, height),
    (width, height),
    (width, int(height * 0.55)),
    (int(width * 0.5), int(height * 0.55)),
    (0, int(height * 0.55))
]], dtype=np.int32)

cv2.fillPoly(mask, polygon, 255)

# Keep only edges inside the ROI
roi = cv2.bitwise_and(edges, mask)

# Detect line segments
lines = cv2.HoughLinesP(
    roi,
    1,
    np.pi / 180,
    threshold=50,
    minLineLength=50,
    maxLineGap=20
)

# Copy image for drawing
output = image.copy()

# Draw detected lane candidates
if lines is not None:
    for line in lines:
```

```
x1, y1, x2, y2 = line[0]
cv2.line(
    output,
    (x1, y1),
    (x2, y2),
    (0, 255, 0),
    3
)

cv2.imshow("Lane Detection", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Functions used

- `cv2.fillPoly(img, pts, color)`
- `cv2.bitwise_and(src1, src2, mask=mask)`
- all preprocessing/Hough functions above.

What ROI means

ROI = Region of Interest. It restricts processing to the part of the image where lane markings are expected.

Main exam trigger

Lane -> edges -> Hough lines

Task 7 - Coin Detection and Counting

What the task wants

Use Hough Circle Transformation to identify and count coins of different sizes and positions.

Workflow

```
Coin image
↓
Grayscale
↓
Gaussian Blur
↓
HoughCircles
↓
Detected circles
↓
len(circles[0])
↓
Coin count
```

Complete solution

```
import cv2
import numpy as np

# Read coin image
image = cv2.imread("coins.jpg")

# Convert to grayscale
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

# Smooth to reduce noise
blurred = cv2.GaussianBlur(gray, (9, 9), 2)

# Detect circular coins
circles = cv2.HoughCircles(
    blurred,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)

# Count and draw circles
```

```
if circles is not None:

    circles = np.uint16(
        np.around(circles)
    )

    count = len(circles[0])
    print("Number of coins:", count)

    for x, y, r in circles[0]:
        # Draw outer circle
        cv2.circle(
            image,
            (x, y),
            r,
            (0, 255, 0),
            2
        )

        # Draw center point
        cv2.circle(
            image,
            (x, y),
            2,
            (0, 0, 255),
            3
        )

    else:
        print("Number of coins: 0")

cv2.imshow("Coin Detection", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Functions used

- cv2.HoughCircles
- np.around
- np.uint16
- len

- `cv2.circle`
- `cv2.imshow`
- `cv2.waitKey`
- `cv2.destroyAllWindows`

Main exam parameters

- `minDist` -> prevents multiple detections too close together.
- `param1` -> internal Canny-related threshold.
- `param2` -> circle acceptance threshold.
- `minRadius` , `maxRadius` -> size range.

Task 8 - Smart Security System / Boundary Detection in Video

What the task wants

Monitor a predefined security zone in real-time video, detect object boundaries, and trigger an alert when an object enters the zone.

The exact boundary/object method is not prescribed in the task statement, so this implementation uses the Lab 06 boundary/edge material plus contours.

Workflow

```
Video
↓
Read frame
↓
Grayscale
↓
Blur
```

↓
Canny
↓
Contours
↓
Bounding boxes
↓
Predefined security zone
↓
Check object position
↓
ALERT

Complete solution

```
import cv2

# Open security video
cap = cv2.VideoCapture("security_video.mp4")

while True:

    # Read next frame
    ret, frame = cap.read()

    # Stop when the video ends
    if not ret:
        break

    # Convert frame to grayscale
    gray = cv2.cvtColor(
        frame,
        cv2.COLOR_BGR2GRAY
    )

    # Reduce noise
    blurred = cv2.GaussianBlur(
        gray,
        (5, 5),
        0
    )

    # Detect boundaries/edges
```

```
edges = cv2.Canny(
    blurred,
    50,
    150
)

# Find external contours
contours, _ = cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

# Get frame dimensions
h, w = frame.shape[:2]

# Define a rectangular security zone
zone_x1 = int(w * 0.25)
zone_y1 = int(h * 0.25)
zone_x2 = int(w * 0.75)
zone_y2 = int(h * 0.75)

# Draw the security zone
cv2.rectangle(
    frame,
    (zone_x1, zone_y1),
    (zone_x2, zone_y2),
    (255, 0, 0),
    2
)

alert = False

# Examine detected boundaries
for contour in contours:

    # Ignore tiny contours/noise
    area = cv2.contourArea(contour)
    if area < 500:
        continue

    # Get bounding rectangle
    x, y, bw, bh = cv2.boundingRect(contour)

    # Compute object center
    center_x = x + bw // 2
```

```

center_y = y + bh // 2

# Draw object bounding box
cv2.rectangle(
    frame,
    (x, y),
    (x + bw, y + bh),
    (0, 255, 0),
    2
)

# Check whether object center is inside security zone
if (
    zone_x1 <= center_x <= zone_x2
    and
    zone_y1 <= center_y <= zone_y2
):
    alert = True

# Trigger visible alarm
if alert:
    cv2.putText(
        frame,
        "ALERT!",
        (50, 50),
        cv2.FONT_HERSHEY_SIMPLEX,
        1.2,
        (0, 0, 255),
        3
    )

# Show real-time result
cv2.imshow(
    "Security System",
    frame
)

# Press q to quit
if cv2.waitKey(1) & 0xFF == ord("q"):
    break

# Clean up
cap.release()
cv2.destroyAllWindows()

```

Function / argument dictionary

`cv2.findContours(image, mode, method)`

- `image` = binary/edge image.
- `mode` = contour retrieval mode; here `cv2.RETR_EXTERNAL` keeps external contours.
- `method` = contour approximation method; here `cv2.CHAIN_APPROX_SIMPLE`.
- returns contours and hierarchy.

`cv2.contourArea(contour)`

- `contour` = one contour.
- returns contour area.

`cv2.boundingRect(contour)`

- `contour` = contour to enclose.
- returns `(x, y, width, height)`.

`cv2.rectangle(img, pt1, pt2, color, thickness)`

- `img` = image.
- `pt1` = top-left corner.
- `pt2` = bottom-right corner.
- `color` = BGR tuple.
- `thickness` = border thickness.

`cv2.putText(img, text, org, fontFace, fontScale, color, thickness)`

- `img` = image.
- `text` = text to display.
- `org` = starting position.
- `fontFace` = font constant.
- `fontScale` = text size.

- `color` = BGR tuple.
- `thickness` = text thickness.

Main exam recognition

Security zone -> video -> edges -> contours -> bounding box -> inside zone? -> alert

PART E - MASTER FUNCTION REFERENCE

Image / color / display

```
cv2.imread(filename, flags=None)
```

Reads image.

```
cv2.cvtColor(src, code)
```

Common Lab 06 codes:

- `cv2.COLOR_BGR2GRAY`
- `cv2.COLOR_BGR2RGB`

```
cv2.imshow(windowName, image)
cv2.waitKey(delay)
cv2.destroyAllWindows()
```

Display image / wait / close windows.

Smoothing / edges

```
cv2.GaussianBlur(src, ksize, sigmaX)
```

Noise reduction.

```
cv2.Canny(image, threshold1, threshold2)
```

Edge map.

```
cv2.Sobel(src, ddepth, dx, dy, ksize)
```

Directional gradient.

```
cv2.Laplacian(src, ddepth)
```

Second derivative.

```
cv2.convertScaleAbs(src, alpha=1, beta=0)
```

Display-friendly absolute 8-bit conversion.

Hough

```
cv2.HoughLinesP(image, rho, theta, threshold, minLineLength, maxLineGap)
```

Line segments.

```
cv2.HoughCircles(image, method, dp, minDist, param1, param2, minRadius, maxRadius)
```

Circles.

```
cv2.line(img, pt1, pt2, color, thickness)
```

Draw line.

```
cv2.circle(img, center, radius, color, thickness)
```

Draw circle.

SIFT / matching / geometry

```
cv2.SIFT_create()
```

Create SIFT detector.

```
sift.detectAndCompute(image, mask)
```

Returns keypoints + descriptors.

```
cv2.BFMatcher()
```

Create brute-force matcher.

```
bf.knnMatch(queryDescriptors, trainDescriptors, k)
```

Find k nearest descriptor matches.

```
cv2.findHomography(srcPoints, dstPoints, method, ransacReprojThreshold)
```

Estimate transformation and, with RANSAC, reject outliers.

```
cv2.perspectiveTransform(points, H)
```

Transform points using homography.

```
cv2.warpPerspective(src, H, dsize)
```

Warp an image using homography.

```
cv2.polylines(img, pts, isClosed, color, thickness)
```

Draw polygon/polyline, useful for object boundary.

Video / boundaries

```
cv2.VideoCapture(source)
```

Open video/camera.

```
cap.read()
```

Return (ret, frame) .

```
cv2.findContours(image, mode, method)
```

Find boundaries/contours.

```
cv2.contourArea(contour)
```

Contour area.

```
cv2.boundingRect(contour)
```

Bounding rectangle (x, y, w, h) .

```
cv2.rectangle(img, pt1, pt2, color, thickness)
```

Draw rectangle.

```
cv2.putText(img, text, org, fontFace, fontScale, color, thickness)
```

Draw text.

NumPy helpers

```
np.sqrt(a)
np.clip(a, min_value, max_value)
np.uint8(a)
np.uint16(a)
np.float32(a)
np.int32(a)
np.around(a)
np.pi
np.where(condition)
np.std(a)
np.median(a)
np.abs(a)
```

Most useful Lab 06 meanings:

- `np.sqrt` -> gradient magnitude calculation.
- `np.clip` -> restrict values to a range.
- `np.uint8` -> 8-bit image values.
- `np.around` -> round circle coordinates/radii.
- `np.where` -> locate anomaly indices.
- `np.pi` -> Hough angle conversion.

Matplotlib helpers used by manual

```
plt.figure(figsize=(w, h))
plt.subplot(rows, cols, index)
plt.imshow(image, cmap="gray")
plt.plot(data)
plt.scatter(x, y)
plt.title("...")
plt.axis("off")
plt.legend()
plt.tight_layout()
plt.show()
```

PART F - QUICK DECISION MAP FOR THE EXAM

What does the question ask me to find?

EDGE / boundary only

-> Canny / Sobel / LoG

STRAIGHT LINE / lane / screen border

-> Canny -> HoughLinesP

CIRCLE / coin

-> Blur -> HoughCircles

KNOWN OBJECT in another image

-> SIFT -> descriptors -> matching -> RANSAC/Homography -> box

KNOWN OBJECT in VIDEO

-> VideoCapture -> frame -> SIFT -> matching -> Homography -> box

PANORAMA / stitching

-> SIFT -> matching -> Homography -> warpPerspective -> stitch

SENSOR ANOMALY / denoising

-> Wavelet -> denoise -> reconstruct -> residual -> threshold

OBJECT BOUNDARY IN VIDEO / SECURITY ZONE

-> frame -> Canny -> contours -> boundingRect -> zone check -> alert

PART G - DO NOT MIX THESE UP

Sobel vs Canny vs LoG

Sobel -> directional gradient (G_x , G_y)

Canny -> multi-stage edge detector with hysteresis

LoG -> Gaussian + Laplacian / second derivative

Hough Line vs Hough Circle

HoughLinesP -> straight line segments
HoughCircles -> circular shapes

Keypoint vs Descriptor

Keypoint -> feature location / local feature point
Descriptor -> numbers describing that feature for matching

Matching vs Homography vs RANSAC

Matching -> finds candidate corresponding features
RANSAC -> rejects geometrically inconsistent matches
Homography -> estimates the geometric mapping between images

Homography vs warpPerspective

findHomography -> FIND the transformation matrix
warpPerspective -> APPLY the transformation to the image

PART H - 30-SECOND CODING CHECKLIST

1. What is my input: BGR, grayscale, image pair, video, or signal?
2. What does the question actually want me to FIND?
3. Which technique is explicitly required?
4. What input does that technique need?
5. What is the function name and argument order?
6. Which parameter is likely to change?
7. What does the function return?
8. What must I do with the returned result?
9. Do I need to display/save the output?
10. Have I handled None / end-of-video cases when appropriate?

PART I - FAST MCQ MEMORY

Sobel	-> first derivative / gradient
Gx	-> dx=1, dy=0
Gy	-> dx=0, dy=1
Gradient magnitude	-> $\sqrt{Gx^2 + Gy^2}$
Canny	-> low + high hysteresis thresholds
Canny noise reduction	-> Gaussian blur
LoG	-> Gaussian + Laplacian
LoG edge idea	-> zero crossings
HoughLinesP	-> line segments
HoughCircles	-> circles
SIFT_create	-> create SIFT detector
SIFT output	-> keypoints + descriptors
SIFT descriptor	-> 128-D standard descriptor
BFMatcher	-> descriptor matching
knnMatch(k=2)	-> two nearest matches for ratio test
RANSAC	-> outlier rejection / robust geometry
findHomography	-> geometric transformation matrix
perspectiveTransform	-> transform points
warpPerspective	-> warp image
VideoCapture	-> open video/camera
cap.read	-> next frame
findContours	-> object boundaries
boundingRect	-> rectangular box

PART J - HIGHEST-VALUE CHEAT-SHEET SNIPPETS

Sobel

```
sobel_x = cv2.Sobel(gray, cv2.CV_64F, 1, 0, ksize=3)
sobel_y = cv2.Sobel(gray, cv2.CV_64F, 0, 1, ksize=3)
magnitude = np.sqrt(sobel_x**2 + sobel_y**2)
```

Canny

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
edges = cv2.Canny(blurred, 50, 150)
```

LoG

```
blurred = cv2.GaussianBlur(gray, (5, 5), 1.4)
laplacian = cv2.Laplacian(blurred, cv2.CV_64F)
laplacian_abs = cv2.convertScaleAbs(laplacian)
```

Hough Line

```
lines = cv2.HoughLinesP(
    edges, 1, np.pi/180,
    threshold=50,
    minLineLength=50,
    maxLineGap=10
)
```

Hough Circle

```
circles = cv2.HoughCircles(
    blurred,
    cv2.HOUGH_GRADIENT,
    dp=1,
    minDist=30,
    param1=100,
    param2=30,
    minRadius=10,
    maxRadius=100
)
```

SIFT

```
sift = cv2.SIFT_create()  
keypoints, descriptors = sift.detectAndCompute(gray, None)
```

Matching

```
bf = cv2.BFMatcher()  
matches = bf.knnMatch(des1, des2, k=2)  
  
good = []  
for m, n in matches:  
    if m.distance < 0.7 * n.distance:  
        good.append(m)
```

Homography

```
H, mask = cv2.findHomography(  
    src_pts, dst_pts,  
    cv2.RANSAC, 5.0  
)
```

Transform corners

```
box = cv2.perspectiveTransform(corners, H)
```

Panorama

```
warped = cv2.warpPerspective(  
    img2, H, (width, height)  
)
```

Video

```
cap = cv2.VideoCapture("video.mp4")
ret, frame = cap.read()
```

Contours

```
contours, _ = cv2.findContours(
    edges,
    cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

x, y, w, h = cv2.boundingRect(contour)
```

Wavelet anomaly core

```
coeffs = pywt.wavedec(signal, "db4", level=4)
# threshold detail coefficients
denoised = pywt.waverec(coeffs, "db4")
residual = signal - denoised[:len(signal)]
anomalies = np.abs(residual) > threshold
```

FINAL MEMORY RULE

Do NOT memorize eight giant programs.

Memorize the reusable blocks.

QUESTION

↓

TARGET

↓

MAIN TECHNIQUE

↓

PREREQUISITES



OUTPUT ACTION

Examples:

Lane -> Canny -> HoughLinesP
Coins -> HoughCircles
Object -> SIFT -> Match -> Homography -> Box
Panorama -> SIFT -> Match -> Homography -> Warp -> Stitch
Security -> Canny -> Contours -> Zone check
Anomaly -> Wavelet -> Reconstruct -> Residual